Adversarially Robust Face Presentation Attack Detection: Checkpoint Report

Shivam Shelar
Department of Computer Science, San José State University
Project nature: *Build* an adversarially robust face presentation attack detector.

I. Problem Statement and Motivation

Problem. Face presentation attack detection (PAD) decides, from a single RGB face image x, whether the camera sees a live person or a presentation attack (a printed photo, a poster, or a face replayed on a screen). We learn a classifier $f(x) \in [0, 1]$, the probability that x is live, and report the ISO/IEC 30107-3 error rates [1]: APCER (attacks accepted as live), BPCER (live faces rejected), and their mean ACER. The project goal is a detector that keeps low APCER and BPCER not only on clean images but also when an attacker adds a small, bounded perturbation δ ($\|\delta\|_\infty \leq \epsilon$) to a spoof image to make it pass.

Why it matters. Face verification now gates phone unlock, payments, remote identity checks and self-service kiosks, and print or replay spoofs remain the cheapest way to attack it [2]. Learned PAD models are themselves vulnerable to adversarial examples: AdvGen showed that perturbations which survive printing can push spoofs past state-of-the-art PAD systems [3]. A PAD model that is accurate but trivially fooled therefore gives a false sense of security.

Scope changes since the proposal (in blue). (i) We use the print and replay attack types of CelebA-Spoof (photo, poster, A4 print, PC, pad and phone screens) and exclude masks, which a kiosk camera would face rarely and which add a different problem. (ii) Training uses a balanced, subject-capped subset (Sec. II) rather than all 625K images, to fit the free GPU budget. (iii) The attack remains digital (PGD [4]) as the stand-in for AdvGen's physical attack. (iv) Preliminary experiments showed that robustness at the common $8/255$ budget is not achievable here (Sec. IV-E), so the defense now targets small budgets ($\leq 2/255$) and every model is evaluated with a sweep over $\epsilon \in \{1, 2, 4, 8, 16\}/255$. (v) The trajectory regularizer adapted from [5] is kept only as an ablation. (vi) The webcam demo moves to the final milestone.

II. Data Engineering

Origin. CelebA-Spoof [6]: 625,537 images of 10,177 subjects with 43 attributes per image (40 face attributes plus spoof type, illumination and environment) and a face bounding box per image. It is distributed for non-commercial research only; we use the Kaggle copy (`attentionlayer241/celeba-spoof-for-face-antispoofing`, about 753K files, 58.8 GB) and its official intra-test protocol, whose train and test subjects are intended to be disjoint. No data is committed to the repository; the README gives the download and preparation commands.

Characteristics. The copy has 8,192 training and 1,004 test subject folders. Labels: live or spoof, spoof type (1–10), illumination (normal, strong, back, dark), environment (indoor, outdoor). Source images have varied resolutions; every model input is a 224×224 face crop (below).

Cleaning and preprocessing. The script `data/prepare_celeba_spoof.py` applies five steps: (1) keep only labelled images present on disk and only the six print/replay types; (2) crop a square of $1.5\times$ the annotated face box, because the background (screen bezel, paper edge) carries attack evidence but the full frame is dominated by scene content; (3) *reject* crops that run outside the photo: on our first load, zero-padded borders appeared in 33% of live but 60% of spoof crops, a shortcut a CNN would exploit; after the fix, dark-edge rates fell to 2.3% and 5.3%; (4) take at most 8 images per subject per class, which spreads the same budget over about four times as many people (1,814 training and validation subjects instead of about 500); (5) resize to 224×224 and normalize with ImageNet statistics; training adds horizontal flips and colour jitter. Sampling is deterministic (sorted listings, seed 42). No hand-made features or PCA are used: the CNN learns directly from pixels, and Sec. III shows simple image statistics do not separate the classes.

Schema sanity and quality. No sample lacked a face box. Besides the padding artifact above, we checked subject leakage (none between train and validation; one subject ID, of 384, appears in both CelebA-Spoof's train and test folders, inherited from the source split) and near-duplicates via 16×16 image hashes (5 groups, all within one split, none with conflicting labels). No crop lacked labels or metadata. Shortcomings for our problem: one dataset only; spoofs are re-captures of celebrity photos; and the test split's spoof-type mix differs from training (Fig. 1), which we return to in Sec. IV.

Split. Table I. Validation subjects are a random 15% of training subjects; test comes from the official test split. Classes are balanced by construction.

III. Exploratory Data Analysis

All figures are produced by `evaluation/eda.py`.

TABLE I
PREPARED FACE CROPS (LIVE / SPOOF).

Split	Crops per class	Subjects	Source
Train	2,550 / 2,550	1,552	CelebA-Spoof train
Val	450 / 450	262	held-out train subjects
Test	600 / 600	384	CelebA-Spoof test

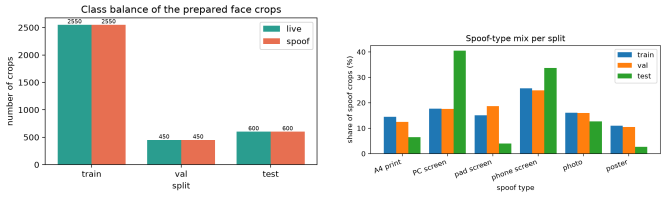


Fig. 1. Class balance and spoof-type mix per split. Classes are balanced, but PC-screen replays are 40.5% of test spoofs against 17.6% of training spoofs, a shift that later explains most baseline errors.

Class balance (Fig. 1). Equal class counts mean accuracy is not misleading, but because the two error types have different costs we always report APCER and BPCER separately, with counts. The spoof-type imbalance between train and test motivates per-type error analysis rather than oversampling, since the test mix is fixed by the protocol. **Univariate** (Fig. 2). Brightness is nearly identical (mean 113 live vs 109 spoof); spoofs are less saturated (93 vs 71) and less sharp ($\log_{10}$ Laplacian variance 2.52 vs 2.15), as expected from re-capture through a print or a screen, yet the box plots overlap over most of their range. **Bivariate** (Fig. 3). Correlations with the label are moderate at best (sharpness 0.40, saturation 0.35, contrast 0.20, high-frequency energy 0.19, brightness 0.06). Sharpness and high-frequency energy are correlated ($r = 0.60$), so they carry overlapping texture information. **By type** (Fig. 4). Printed photos sit closest to live on these statistics, and the sharpest PC-screen replays overlap live faces. Together these results justify a learned model and predict the baseline's failure on sharp screen replays (Sec. IV-D).

IV. BASELINE AND PRELIMINARY RESULTS

A. Setup

ResNet-18 [7] with ImageNet weights and a single output logit, trained with binary cross-entropy, AdamW (lr 10^{-4}), batch 32, 5 epochs; the epoch with the best validation accuracy is kept. Three seeds (0, 1, 2) fix all randomness (`utils/seed.py`); the commit hash is saved with the results.

B. Metrics with counts

Table II. Real faces are almost always accepted (BPCER about 1%), but 22–39% of attacks pass, and the spread across seeds (135 to 232 false accepts) shows that the baseline's behaviour on the shifted test distribution is unstable, while validation is nearly perfect for every seed. Seed 0 reproduces the earlier pilot run on CUDA exactly in false accepts (135/600).



Fig. 2. Univariate image statistics (brightness, contrast, saturation, sharpness, high-frequency energy) as histograms (top) and box plots (bottom), live vs spoof. Spoofs are on average less saturated and less sharp, but every distribution overlaps heavily, so no single statistic separates the classes.

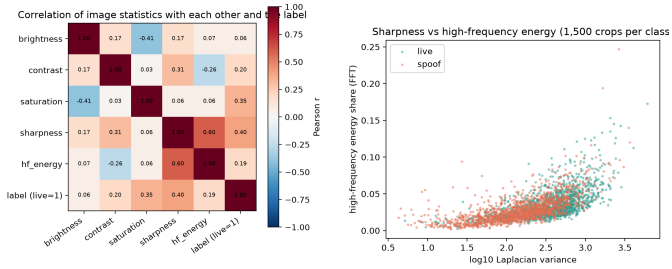


Fig. 3. Left: Pearson correlation of the statistics with each other and with the label; no statistic exceeds $|r| = 0.40$ with the label. Right: sharpness against high-frequency energy, where spoofs spread wider but overlap live faces almost everywhere.

C. Threshold analysis

We choose the operating point on *validation* (its EER threshold) and apply it unchanged to test, so the test set never tunes the model (Fig. 5). For seed 0 this threshold is 0.304, giving APCER 27.2% (163/600) and BPCER 1.3% (8/600) on test. Because validation contains almost no errors, its EER threshold is poorly determined (0.30, 0.79 and 0.26 for the three seeds) and does not transfer: on test it raises APCER slightly instead of balancing the two errors (test EER is 10.6%). For a kiosk, a false accept is costlier than asking a real user to retry, so the preferred point is a high threshold; on test, a threshold near 0.9 would bring APCER toward 10% at a BPCER near 8% (Fig. 5), but choosing it honestly needs a validation set that contains the hard screen replays. We report the validation-chosen point and treat this as a finding.

D. Error analysis

Errors concentrate in one attack type (Table III): PC-screen replays account for 127 of the 135 false accepts, while every training fake, PC screens included, is rejected. Failures also cluster in normal lighting (29.8% accepted) and indoor scenes (25.9%), against 0–3% under strong, back or dark light and 2.3% outdoors. Inspecting the accepted PC-screen crops shows sharp, well-lit captures with no visible pixel grid or moiré, while correctly rejected ones show that texture. **Hypothesis:** the baseline relies on fine screen texture, which disappears in high-quality captures; the test split contains many more of these than training, which also explains the gap between

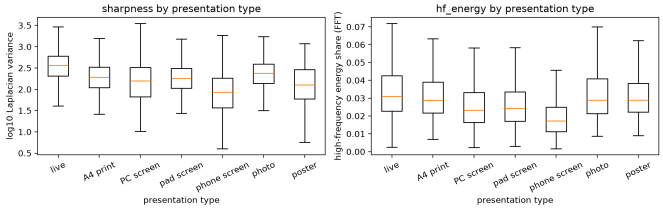


Fig. 4. Sharpness and high-frequency energy by presentation type. Phone screens are the blurriest; PC screens have the widest spread, and their sharp upper tail overlaps live faces.

TABLE II
BASELINE ON THE TEST SET (600 LIVE, 600 SPOOF). MEAN ± STD OVER 3 SEEDS; COUNTS (OUT OF 600) PER SEED.

	Default (p=0.5)	Operating point
Threshold	0.500	0.45 ± 0.29
APCER	$28.8 \pm 8.7\%$	$31.3 \pm 3.6\%$
seed 0 / 1 / 2	135 / 232 / 151	163 / 204 / 196
BPCER	$1.3 \pm 0.9\%$	$1.3 \pm 0.4\%$
seed 0 / 1 / 2	10 / 2 / 12	8 / 10 / 5
ACER	$15.1 \pm 3.9\%$	$16.3 \pm 1.8\%$
EER (val / test)	$1.3 \pm 0.4\%$ / $10.6 \pm 2.5\%$	
AUC (val / test)	0.999 ± 0.000 / 0.963 ± 0.013	

validation and test. The most confident false accepts (Fig. 6) repeat a few subjects, so the 135 errors are not independent: several come from re-captures of the same person. Among the eight rejected real faces, several are themselves photos of prints or magazine pages in the source data, so part of the BPCER reflects label ambiguity rather than model error.

E. Preliminary robustness results

A 10-step PGD attack flips 100% of the baseline's test fakes at every budget from $1/255$ to $16/255$. Adversarial training at $2/255$ lowers this: among fakes the hardened model rejects without attack, a $2/255$ attack flips 29% (11% at $1/255$), reproduced across two training schemes, at the cost of a higher clean APCER (about 42% vs 22%), concentrated on screen replays. Training at $8/255$ with up-weighted fakes appeared to give 2% attack success, but the budget sweep exposed it: smaller attacks still succeed 98% of the time, which is impossible for a truly robust model and indicates gradient masking [8].

Reproducibility. Every number and figure in Secs. II–IV-D comes from one run of `scripts/run_checkpoint_mac.sh` (equivalently `notebooks/kaggle_checkpoint.ipynb`) at commit `b20209a`, on an Apple M-series GPU (PyTorch MPS); results on CUDA differ slightly. The robustness numbers in Sec. IV-E are preliminary single-seed Kaggle runs. The README maps each script to its section.

REFERENCES

[1] ISO/IEC, "ISO/IEC 30107-3: Information technology – biometric presentation attack detection – part 3: Testing and reporting," 2017.

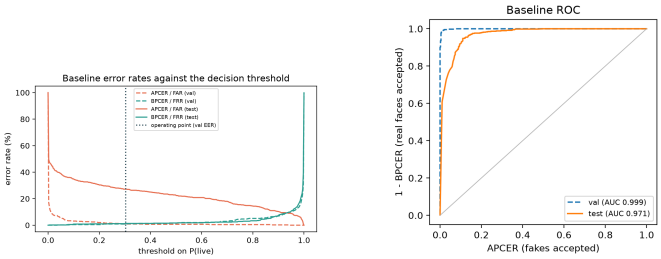


Fig. 5. Left: APCER (FAR) and BPCER (FRR) against the threshold on validation and test, with the chosen operating point (seed 0). Right: ROC curves. Validation is almost perfectly separable (AUC 0.999), while on test the APCER curve stays high over most thresholds.

TABLE III
TEST SPOOFS ACCEPTED AS LIVE BY SPOOF TYPE (SEED 0, p=0.5).

Spoof type	Accepted / total	Rate
PC screen	127 / 243	52.3%
Poster	5 / 16	31.2%
Photo	2 / 76	2.6%
Phone screen	1 / 202	0.5%
A4 print	0 / 39	0.0%
Pad screen	0 / 24	0.0%
All	135 / 600	22.5%

[2] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, 2023.

[3] S. A. Patnaik, S. Chansoriya, A. K. Jain, and A. M. Namboodiri, "AdvGen: Physical adversarial attack on face presentation attack detection systems," in *IEEE International Joint Conference on Biometrics (IJCB)*, 2023.

[4] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations (ICLR)*, 2018.

[5] Y. Sun, Y. Liu, X. Liu, Y. Li, and W.-S. Chu, "Rethinking domain generalization for face anti-spoofing: Separability and alignment," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] Y. Zhang, Z. Yin, Y. Li, G. Yin, J. Yan, J. Shao, and Z. Liu, "CelebA-Spoof: Large-scale face anti-spoofing dataset with rich annotations," in *European Conference on Computer Vision (ECCV)*, 2020.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," in *International Conference on Machine Learning (ICML)*, 2018.

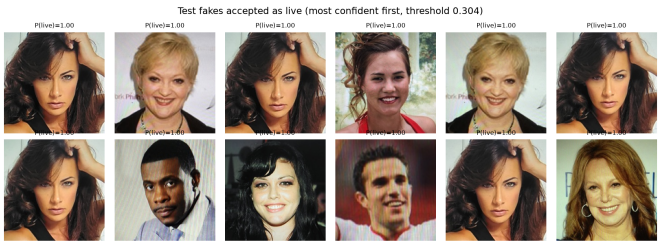


Fig. 6. The 12 most confident test false accepts (seed 0, operating point). Most are sharp PC-screen re-captures, and the same subjects recur.